

Unit- 1 Modules and packages

Modules :

Modules **refer to a file** containing **Python statements and definitions**.

We use modules **to break down large programs into small manageable and organized files**.

Furthermore, modules provide **reusability of code**.

We can define our most used functions in a module **and import it**, instead of copying their definitions into different programs.

- Modules are of two types

1) Pre-defined Module

2) User defined Module

1) Pre defined Module: The module which is **already defined by Python**, is called as Pre defined Module
e.g. math, random, time , etc. module

To use predefined module firstly we have **to import** it as,

import modulename

where, import is a keyword

After this we can use the methods of module by **calling them** as,

Modulename.methodname(arg1,arg2,...)

There are large number of modules present in python library we are going to understand few of them.

a) math Module:

Python has a built-in module called math module that can **use for mathematical tasks**.

The math module contains many predefined functions to **perform many mathematical operations**. Let's see some of methods present in math module .

* **sqrt() method**: This method is present in math module to **find square root of a number**.

```
import math
print(math.sqrt(4))
```

Output: 2.0

* **cos() method** : is used to find cosine.

```
import math
print(math.cos(0))
```

Output: -1

Also some constants defined under math module **pi** is one of them

```
import math
print (math.pi)
```

Output : 3.141

Like this many more methods and constants present in math module.

Time module : Python has a module named **time** , to **handle time-related tasks**.

***sleep() method**: The sleep() function **suspends (delays) execution of the current thread** for the given number of seconds.

```
import time
print("This is printed immediately.")
time.sleep(2.4)
print("This is printed after 2.4 seconds.")
```

Output: This is printed immediately. This is
printed after 2.4 seconds.

Like this time module have many more methods.

b) random module : Python has a built-in module that we can use to **make random numbers**.

***shuffle() Method:** Shuffle a list (**reorganize the order** of the list items):

```
import random
mylist = ["apple","banana","cherry"]
random.shuffle(mylist)
print(mylist)
```

Output : ['cherry', 'banana', 'apple']

Like this random module also have many more methods .

2) User defined module:

The module which is **defined by User** , is called as User defined Module.

Let us **create a User defined module**. Type the following and save it as **example.py**

```
def add(a, b):
    result = a + b
    return result
```

Here example is our module name and We defined add method in our module Which will be accessed in further Programs.

Now let's **access our add() method** of **example** module to do addition in our program as,

```
import example
print(example.add(4,3))
```

Output: 7

* Import with renaming :

We can import a module by **renaming it** as follows:# import module by renaming it

```
import math as m
print("The value of pi is", m.pi)
```

Output : 3.141

* Python from...import statement

We can **import specific names** from a module without importing the module as a **whole**. Here is an example.

```
from math import pi
print("The value of pi is", pi)
```

Package :-

A Package is a **collection of different modules with an `__init__.py` file.**

A package is a container that **contains various functions to perform specific tasks.**

For example, the math package includes the `sqrt()` function to perform the square root of a number.

Python packages are a **way to organize and structure code by grouping related modules into directories.**

A package is essentially a folder that contains an `__init__.py` file and one or more Python files (modules). This organization helps manage and reuse code effectively, especially in larger projects. It also allows functionality to be easily shared and distributed across different applications. Packages act like toolboxes, storing and organizing tools (functions and classes) for efficient access and reuse.

Key Components of a Python Package

- **Module:** A single Python file containing reusable code (e.g., `math.py`).
- **Package:** A directory containing modules and a special `__init__.py` file.
- **Sub-Packages:** Packages nested within other packages for deeper organization.
-

How to create and access packages in python

1. **Create a Directory:** Make a directory for your package. This will serve as the root folder.
2. **Add Modules:** Add Python files (modules) to the directory, each representing specific functionality.
3. **Include `__init__.py`:** Add an `__init__.py` file (can be empty) to the directory to mark it as a package.
4. **Add Sub packages (Optional):** Create subdirectories with their own `__init__.py` files for sub packages.
5. **Import Modules:** Use dot notation to import, e.g., `from mypackage.module1 import greet`.

Ex.

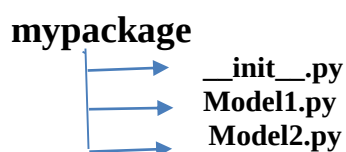
1) `__init__` :
can be empty

2) **Model1.py:-**
`def show():`
`print("This is in model1")`

3) **Model2.py:-**
`def add(a,b):`
`return(a+b)`

4) **package.py :-**

```
import mypackage.model1
import mypackage.model2
mypackage.model1.show()
x=mypackage.model2.add(20,30)
print(x)
```



Object Oriented Programming :-

Features of object-oriented programming in Python:-

- 1)Encapsulation:** This feature allows you **to group data and functions into a single unit** called a class. The data is **hidden from the outside world**, and only the methods or functions defined in the class can access it.
- 2)Inheritance:** Inheritance is a way **to create a new class from an existing class**. The new class inherits the properties and behavior of the existing class and can also add its own properties and behavior.
- 3)Polymorphism:** Polymorphism is the **ability of an object to take on many forms**. In Python, polymorphism is achieved through method overloading and method overriding.
- 4)Abstraction:** Abstraction is **the process of hiding unnecessary details** while showing only the essential features of an object. It helps in reducing complexity and making code easier to understand and maintain.
- 5)Class:** A class is a **blueprint or template that defines the properties and behavior of an object**. It contains variables (called attributes) and functions (called methods) that can be used to manipulate the object.
- 6)Object:** An **object is an instance of a class**. It contains data (attributes) and functions (methods) that operate on that data.
- 7)Method:** A method is a **function defined in a class that can be used to manipulate the object**.
- 8)Instance:** An instance is a **unique copy of an object** that is created from a class.

Class:

A class is user defined data type which is **collection of variables and methods**. These variables are called ‘properties or attributes’ and methods are called ‘actions’. All variables and methods are also available in objects, because they are created from class therefore they are called ‘instance variable’ and ‘instance method’.

Class Definition Syntax:

Where,

class: is a keyword

Classname: should be an identifier

```
class ClassName:  
    # Statement-1  
    .  
    .  
    # Statement-N
```

Object:

An Object is **an instance of a Class**. A class is like a blueprint while an instance is a copy of the class with actual values.

An object consists of :

State : It is represented by attributes of an object. It also reflects the properties of an object.

Behavior : It is represented by methods of an object. It also reflects the response of an object with other objects.

Identity : It gives a unique name to an object and enables one object to interact with other objects.

Syntax to create an object

```
objectname=Classname()
```

Example

```
s=Student()
```

A simple program

```
class Student:
    roll=10      #variable
    nm="ABhi"    #variable

    #Method
    def show(self):
        print("Roll Number-",self.roll)
        print("Name-",self.nm)

s=Student()    #object creation
s.show()       #call to method
```

In python instance methods of a class contains one compulsory argument called **self** which refers memory address of current invoking object.

In above program we accessed variables by using self keyword.

Also show() method is called using help of object.

Variables :

We know that variable is a **name provided to memory location**, Where data is saved.

Types of variable-

- a) instance variable
- b) class or static variable

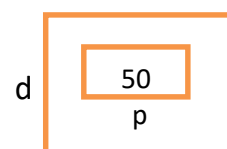
a) Instance variable: Variables which are **declared inside the constructor and method** of class are called as Instance variables.

They are **tied to the particular object** instance of the class.

```
class Demo:
    def dis(self):
        self.p=50    # instance variables
        print("Value of p",self.p)

d=Demo()    #object creation
d.dis()     #method call
```

Output: Value of p 50



In above program p is a instance variable because it is declared in dis() method.

p variable is accessed within that method by using self as **self.p** because all instance variables are of a class is accessed as **self.variablename** .

b) Class or static variable:

A variable which is **declared inside the class definition (but outside any of the instance methods)** are called as class variables.

They are **not tied to any particular object of the class**, hence shared across all the objects of the class.

Modifying a class variable affects all objects instance at the same time.

```
class Demo:
    v=100          # class variables
    def dis(self):
        print("Value of v",self.v)

d=Demo() #object creation
d.dis()  #method call
```

Output: Value of v 100

In above program variable v is declared in class Demo but it is defined outside dis() method so It is called as **class variable**.

Class Variables are also accessed by using object of that class in which it is declared.

Self variable :

Self is a **default variable that contains memory address of the instance of the current class**.

So we can use **self to access** instance variables and instance methods.

The self keyword is used as **first parameter of a method or first parameter of constructor**

E.g.

```
def show(self)
```

Methods:

A method can access and modify the data attributes of the class and perform specific task on the instance.

A method in python is somewhat **similar to a function, except it is associated with object/classes**.

There are 3 types of methods in python

A) Instance method B) Class method C)static method

A) Instance method :

Instance method is a very basic and **easy method that we use regularly** when we create classes inpython.

If we want to **call the instance method**, we must **create an object** of that required class and we should **call the instance method by using that object**.

The **self** should be used as a first argument of instance method.

Syntax for method

```
class ClassName:
    def method_name(self):
        # Method_body
```

Example of Instance method

```
class A:
    def add(self):
        self.a=10
        self.b=10
        self.c=self.a+self.b
        print("Addition is",self.c)

x=A() #object creation
x.add() #method call
```

Output: Addition is 20

In above program we defined add() instance method which contains **self** as first argument.

Self used to access instance variables inside add() method.

We called add() method by using object **x** because instance method is called using object of that class.

B) Class Method :

- A class method is a method which **is bound to the class and not the object of the class.**
- They have the access to the state of the class as **it takes a `cls` (class) parameter** that points to the class **and not the object instance.**
- It can modify a class state that would apply across all the instances of the class. For example it can modify a class variable that will be applicable to all the instances.

The **@classmethod decorator**, is a builtin [function decorator](#) that is an expression that gets evaluated after your function is defined.

syntax

```
class classname:
    @classmethod
    def fun(cls, arg1, arg2, ...):
        body
```

Class method example :

```
class Demo:
    v=100

    @classmethod
    def show(cls):
        return (cls.v)

d=Demo()
print("call using object",d.show()) #call to class method
print("call using class name",Demo.show()) #call to class method
```

c)Static Method :

A static method is a method which is marked with a **@staticmethod decorator** to flag it as static.

It does not receive an implicit first argument (neither self nor cls).

static methods can neither modify the object state nor class state. They are primarily a way to namespace our methods.

Call to static method is generated by its class name as,
Classname.methodname(arg1,arg2,...)

Syntax to define static method

```
class Classname:
    @staticmethod
    def fun(arg1, arg2, ...):
        body
```

Example of static method

```
class Demo:
    @staticmethod
    def add():
        a=12
        b=5
        c=a+b
        print("addition is",c)

Demo.add() #call to static method by using class name
```

Output:

addition is 17

Constructor :

A constructor is a special type of method (function) which is used to **initialize the instance members of the class**.

In C++ or Java, the constructor has the same name as its class, but it **treats constructor differently in Python**.

In Python the `__init__()` method is called the **constructor** and is **always called when an object is created**.

Syntax of constructor definition:

```
Def       init      (self):  
    # body of the constructor
```

There are two types of constructor in python

- 1) Default Constructor
- 2) Parameterised constructor

1) Default constructor :-

The default constructor is simple constructor which **doesn't accept any arguments**.

It's definition has **only one implicit argument which is self** (a reference to the instance being constructed).

```
class Demo:  
    def __init__(self):          #default constructor  
        self.a=10  
        self.b=5  
        self.c=self.a+self.b  
        print("addition is",self.c)  
d=Demo() #call to default constructor
```

Output : addition is 15

2) Parameterised constructor :-

The **constructor with parameters** is known as parameterized constructor.

The parameterized constructor take **its first argument as self** and the **rest of the arguments are provided by the programmer**

```
class Demo:  
    def __init__(self,x,y):      #parameterised constructor  
        self.a=x  
        self.b=y  
        self.c=self.a+self.b  
        print("addition is",self.c)  
  
d=Demo(10,20) #call to default constructor
```

Output:

addition is 30

In above program we took **two explicit formal parameters** x and y in our constructor so that the constructor called as **parameterised constructor**.

We passed two values 10 and 20 to x and y from actual parameters present in constructor call.

Constructor with default Arguments :

The constructor which contains **formal parameters with default assigned values** is called as constructor with default arguments.

It is nothing but a **parameterised constructor**.

In this concept there is **no need to pass values to this constructor while calling it**.

```
class Demo:
    #constructor with default arguments
    def __init__(self,x=100,y=30):
        self.a=x
        self.b=y
        self.c=self.a+self.b
        print("addition is",self.c)

d=Demo() #call to default constructor
```

Output:
addition is 130

In above program we have a parameterised constructor which contains two formal parameters x and y which are assigned by default values 100 and 30 simultaneously. So our constructor is called as constructor with default arguments .

In this case there is no need to pass values to formal parameters from actual parameters.

So our constructor call is without actual parameters.

Passing members of one class To another Class by using object :

We know that all class data is in object of that class.

We can pass members(variables and methods) of one class to another class for that we can use an object of that class.

```
class Emp:
    def __init__(self):
        self.id=101
        self.name="Nitin"

class Myclass:
    @staticmethod
    def show(e):
        print(e.id)
        print(e.name)

x=Emp()
Myclass.show(x)    #passing object to Myclass
```

Inner classes/ Nested class :

A class defined in another class is known as inner class or nested class.

An outer class can have one or more inner class but generally **inner classes are avoided**.

Syntax to define inner class

```
class outerclassname:
    class Innerclass name:
        body of inner class
```

Syntax to create innerclass object

```
obj=outerclassobject.InnerclassName()
```

Example of inner class

```
class College:
    def __init__(self):
        print("outer class constructor")
        def show(self):
            print("Outer class method")
        #inner class
        class Student:
            def __init__(self):
                print("Inner class constructor")
            def display(self):
                print("Inner class method")
x=College()    #outer class object
y=x.Student() #inner class object
x.show()
y.display()
```

Output:

outer class constructor

Inner class constructor

Outer class method

Inner class method

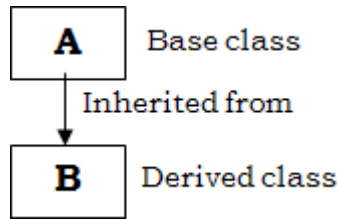
Inheritance :

We know that, inheritance is one of the most important object oriented concept

The main concept behind inheritance **is reusability of code (Software)** is possible by adding some extra feature into existing software without modifying it.

Definition : The mechanism of **creating new class from existing class** is called as Inheritance”.

- The existing or old class is called as ‘**Base class**’ or ‘**Parent class**’ or ‘**Super class**’ and new class is called as ‘**Derived class**’ or ‘**Child class**’ or ‘**Sub class**’.
- While deriving the new class from exiting one then, *the derived class will have some or all the features of existing class and also it can add its own features* i.e sub class will acquires features of base class without modifying it.



In inheritance the base class is defined in normal way as we define our class, but the derived class is defined by following syntax

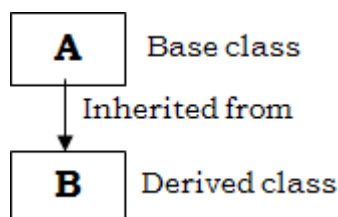
```
class DerivedClassname (BaseClassname):  
  
    Body of derived class
```

Types of Inheritance :

Types of Inheritance depends upon the number of child and parent classes involved. There are five types of inheritance in Python:

- 1) Single Inheritance 2) Multiple Inheritance
- 3) Multilevel Inheritance 4) Hierarchical Inheritance 5) Hybrid Inheritance

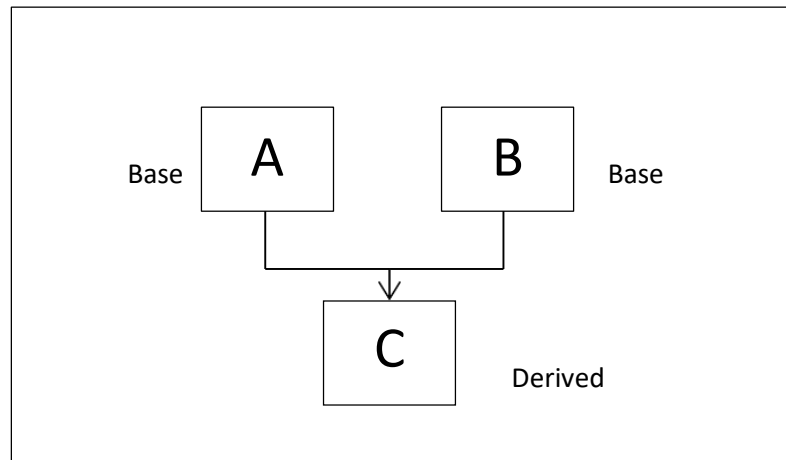
1) Single Inheritance : In case of Single inheritance there is **only one base class from which we derive only one new class.**



```
class A:  
    p=30  
    q=10  
  
class B(A):  
    def show(self):  
        self.r=self.p - self.q  
        print("Subtraction is",self.r)  
  
x=B() #always create derived class object  
x.show()
```

Output- Subtraction is 20

2) Multiple Inheritance : In case of Multiple inheritance there are **multiple**(two or more than two) base classes from which we derive **only one** new class.



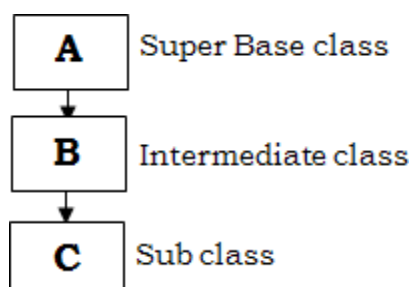
```
class A:
    p=30

class B:
    q=10

class C(A,B):
    def show(self):
        self.r=self.p - self.q
        print("Subtraction is",self.r)

x=C()      #always create derived class object
x.show()
```

3)Multilevel Inheritance :- In case of multilevel inheritance we can **derive new class from another derived classes**, further from derived classes we again derive new class and so on.



Also, there are some levels of inheritances therefore it is called as “Multilevel inheritance”

Example of multilevel Inheritance

```
class A:
    p=30

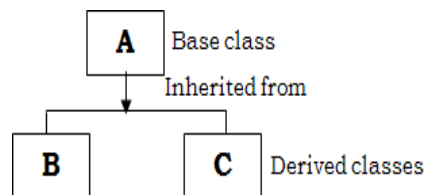
class B(A):
    q=10

class C(B):
    def show(self):
        self.r=self.p*self.q
        print("Multiplication is",self.r)

x=C()    #always create derived class object
x.show()
```

4) Hierarchical Inheritance : In case of hierarchical inheritance there is only **one base class from that class we can derive multiple new classes.**

The base class provides all its features to all derived classes which are common to all sub classes. The hierarchical inheritance comes into picture when certain feature of one level is shared by many other derived classes.



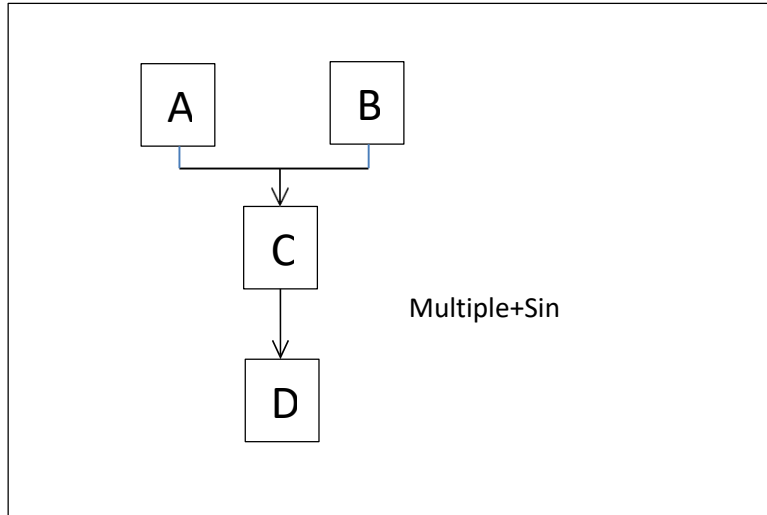
```
class A:
    p=30
    q=10

class B(A):
    def add(self):
        self.r=self.p+self.q
        print("Addition is",self.r)

class C(A):
    def sub(self):
        self.m=self.p-self.q
        print("Subtraction is",self.m)

x=B()    #always create derived class object
x.add()
y=C()    #always create derived class object
y.sub()
```

1) Hybrid Inheritance : Hybrid inheritance is the **combination of two or more forms of inheritances**.



```
class A:
    p=30

class B:
    q=10

class C(A,B):
    r=5

class D(C):
    def show(self):
        self.m=self.p+self.q+self.r
        print("Addition is",self.m)

x=D()      #always create derived class object
x.show()
```

Constructor in Inheritance :

We can add constructor in stayed of methods in classes which are used in Inheritance.

If we **use constructor in base class** then there is **no need to call base class constructor Explicitly**.

When we **create derived class object** then **Base class constructor called automatically**.

Example of constructor in Inheritance :-

```
# Base class
class A:
    def __init__(self):    #base class constructor
        self.a=10
        self.b=5
        print("Sub=",self.a-self.b)

# Derived class
class B(A):
    def show(self):
        print("Addition=", (self.a+self.b))

x=B()    #call to base class constructor
x.show()
```

Output:

Sub=5

Addition= 15

Super() method : The super() builtin returns a **proxy object** (temporary object of the superclass) that allows us to **access methods of the base class**.

In Python, super() has two major use cases:

Allows us to avoid using the base class name explicitly

Working with Multiple Inheritance

```
class college:
    def __init__(self,p):
        print(p)

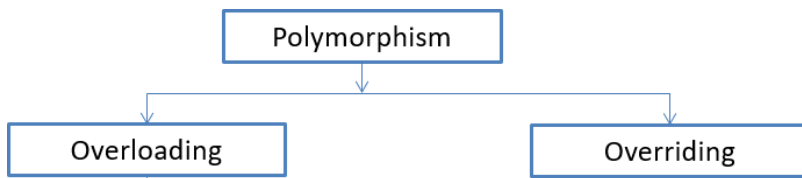
class student(college):
    def __init__(self):
        print('all students are clever')
        super().__init__('BCA')    #call to base class constructor

d1 =student()#call to derived class constructor
```


Polymorphism: Polymorphism is one of the important feature of OOP.

Polymorphism is the **ability of an object to take on many forms** .The word polymorphism **means one name and many forms**.

In python we can implement polymorphism by Overloading and Overriding.



Method Overloading : Method Overloading is the ability of a function to behave in different ways **based on the parameters that are passed to the function**(function call) .

In Python, we can create a method that can be called in different ways. So, we can have a method that has zero, one or more number of parameters. Depending on the method definition, we can call it with **zero, one or more arguments**.

Program of method overloading

```
class Moverload:
    def show(self,a=None,b=None):
        print(a,b)

x=Moverload()
x.show()    #1st call
x.show(10) #2nd call
x.show(5,3)#3rd call
```

Output : None None

10 None

5 3

Constructor Overloading : constructor Overloading is the ability of a constructor(parameterised constructor) to behave in different ways based on the parameters that are passed to the constructor (constructor call) .

```
class Coverload:
    def __init__(self,a=None,b=None):
        print(a,b)

x=Coverload()    #1st call
y=Coverload(10)  #2nd call
z=Coverload(5,3) #3rd call
```

Output:
None None
10 None
5 3

Operator Overloading : Python operators work for built-in classes. But the same operator behaves differently with different types.

For example, the + operator will perform arithmetic addition on two numbers, merge two lists, or concatenate two strings.

Ex. 2+3=5

‘he’+’llo’ -> ‘Hello’

This feature in Python that allows the same operator to have different meaning according to the context is called operator overloading.

In Operator overloading special functions are used which start from __ (double underscore) we can overload other operators as well. The special function that we need to implement is tabulated below.

Operator	Internally
+	__add__(self, other)
-	__sub__(self, other)
*	__mul__(self, other)
/	__truediv__(self, other)
//	__floordiv__(self, other)
%	__mod__(self, other)
**	__pow__(self, other)
>>	__rshift__(self, other)
<<	__lshift__(self, other)
&	__and__(self, other)
	__or__(self, other)
^	__xor__(self, other)

Lets see __add__() function example

When + operator used in between then add function called by int and execute as,

```
print(int.__add__(3,2))
```

Output : 5

When + operator used in between **String** operands then add function called by **str** and execute as,

```
print(str.__add__('he','llo'))
```

Output: Hello

Method Overriding:

Method overriding is a concept of object oriented programming that allows us to **change the implementation of a function in the child class that is defined in the parent class**. It is the ability of a child class to change the implementation of any method which is already provided by one of its parent class(ancestors).

In short we can say that in **method overriding multiple methods in Inheritance having same name and same signature(parameters)**

Program for Method Overriding

```
class A:
    def show(self):
        print("In Base class A")
class B(A):
    def show(self):
        print("In derived class B")
y=B()
y.show()
```

Output :

In derived class B

In method overriding base and derived class methods have same name and same signature. The base class method get overridden by derived class method.

Constructor Overriding (Overriding base class Constructor): When base and derived class **constructors are of same type(same name and same signature)** then that is called as constructor overriding.

In constructor overriding the derived class constructor gets executed.

```
class A:
    def __init__(self):
        print("In Base class A")
class B(A):
    def __init__(self):
        print("In derived class B")
y=B() #call to derived class constructor
```

Output:

In derived class B

In constructor overriding The base class constructor get overridden by derived class constructor.

Abstract Class and Abstract Method:

Abstract Class: A class which **contains one or more abstract methods** is called an abstract class. An abstract class can be considered as a blueprint for other classes. It allows us to create a set of methods that must be created within any child classes built from the abstract class.

- An abstract class doesn't contain all of the method implementations.
- An abstract class cannot be instantiated.
- A derived subclass must implement the abstract methods to create a concrete class that fits the interface defined by the abstract class. Therefore it cannot be instantiated unless all of its abstract methods are overridden.
- To implement abstract class **ABC class** should be imported from **abc module**.
- Our Abstract class should be derived by ABC class as,
- An Abstract class can contain Abstract and concrete methods.

class classname(ABC):

Abstract Method: An abstract method is **a method that is declared, but not implemented in the code**.

The Abstract method should **compulsory redefined in derived classes**.

Before definition of Abstract method the **@abstractmethod** decorator should be declared.

Example of abstract class and Abstract method

```
from abc import ABC,abstractmethod

#Abstract class
class First(ABC):
    @abstractmethod
    def show(self):
        pass    #implement in derived class

class Second(First):
    def show(self):
        self.a=21self.b=33
        self.c=self.a+self.b
        print("Addition",self.c)

d=Second()
d.show()
```

Interface :

A class which **contains only abstract methods** that class is called as Interface.

In python we **cannot create an interface** like created in java.

In other words we say that an abstract class which contains **only abstract** methods that

abstractclass work as an Interface in python.

```
from abc import ABC, abstractmethod

# Interface
class First(ABC):
    @abstractmethod
    def show(self):
        pass

class Second(First):
    def show(self):
        print("In derived class")

d=Second()
d.show()
```

Class method vs static method :

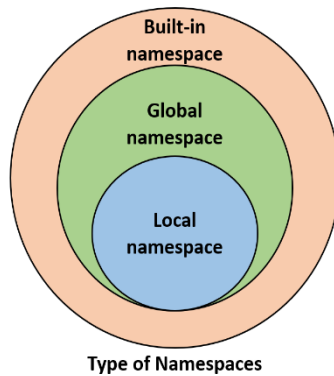
Class Method	Static Method
The class method takes cls (class) as first argument.	The static method does not take any specific parameter.
Class method can access and modify the class state.	Static Method cannot access or modify the class state.
The class method takes the class as parameter to know about the state of that class.	Static methods do not know about class state. These methods are used to do some utility tasks by taking some parameters.
@classmethod decorator is used here.	@staticmethod decorator is used here.

Namespaces :-

A namespace is a collection of objects with unique name.

A namespace is a **collection of names and the details of the objects referenced by the names.**

- Allow the python interpreter to understand which variable or method is referred to code.
- Prevent accidental overwrites of symbols defined in modules.
- ex. Directory -Unique file subdirectory.



➤ **Built-in namespace** – A Python namespace is created when we start using the interpreter, and it exists as long as to use. When the Python interpreter runs without any user-defined modules, methods, etc., built-in functions like `print()` and `id()` are always present and available from any part of a program. These functions exist in the built-in namespace.

➤ **Global namespace** – A global namespace is created when you create a module.

➤ **Local namespace** – This namespace is created when you create new local functions.

A namespace's lifetime depends on the scope of a variable. If the variable's scope ends, then python namespace scope ends.

Behavior of local variables	Using global variables within functions	built-in namespace
<pre>def fun(): a = 4 print(a) fun()</pre>	<pre>a = 10 def fun(): global a print(a) b=4 print(b) fun()</pre>	<pre>a = 0 def fun(): int = 3 input = 4 print(int) print(input) fun()</pre>

Difference between POP and OOP :-

POP	OOP
1) The focus is on function rather than data.	1) The focus is on data rather than function.
2) It follows top-down approach.	2) It follows bottom-up approach
3) Program consists of different Functions.	3) Program consists of different objects.
4) Communication is done between functions.	4) Communication is done between objects
5) Data is not hidden.	5) Data is hidden.
6) New data & functions cannot be added.	6) New data & functions can be added whenever required.
7) COBOL, FORTORN, C etc. are the examples of POP language.	7) C++,Python is the example of OOP language.
8) This is a good programming approach for real time software design	8) This is good for scientific application development
9) It does not supports for the features of OOP's	9) All features of POP may be present in OOP